```
// MyApplication/SubNamespace/SomeOtherClass.php
namespace MyApplication\SubNamespace;

class SomeOtherClass
{
}

// MyApplication/SubNamespace/ThisClassOverHere.php
namespace MyApplication\SubNamespace;

class ThisClassOverHere
{
}
```

Let's tie it all together, like a rug ties a room together:

```
//main.php
$class = new \MyApplication\SomeClass();
var_dump($class);

Output:
LOADING : MyApplication\SomeClass from namespace
LOADING : MyApplication\SubNamespace\SomeOtherClass from namespace
LOADING : MyApplication\SubNamespace\ThisClassOverHere from namespace
object(MyApplication\SomeClass)#2 (2) {
  ["otherClass":"MyApplication\SomeClass":private]=>
  object(MyApplication\SubNamespace\SomeOtherClass)#3 (0) {
  }
  ["thisClassOverHere":"MyApplication\SomeClass":private]=>
  object(MyApplication\SubNamespace\ThisClassOverHere)#4 (0) {
  }
}
```

And there we have it. Namespaced autoloading, PSR-4 style. Notice that when each class is loaded, the value of `__NAMESPACE__` is empty, even when `SomeOtherClass` is instantiated inside the constructor of `SomeClass`. This is because even though we're using namespaced classes, the calling code never left the root namespace (aka "non-namespaced," often called "global" namespace, which is something of a misnomer in the author's humble opinion).